



45-Day DSA Roadmap

Start → Expert · Crack Coding Interviews · Structured & Progressive

Created by Vaibhav Singh (AIML)

PREREQUISITES

- Hands-on programming experience with **Python, Java, or C++**
- Comfortable with basic syntax, loops, conditionals, and functions
- Mindset: **consistency over intensity** — 2–4 hours/day minimum

Insight: If you're starting from zero, don't skip the prerequisites — spend 2-3 extra days getting fluent with your chosen language's standard library (lists, dicts/maps, string methods). It pays back tenfold once you hit patterns.

GROUND RULES — READ BEFORE DAY 1

- ✍️ Write every algorithm by hand first — pen/paper or plain text editor, no IDE
- ⌚ Time yourself: target 20–25 minutes per LeetCode Medium
- 🗣️ Think out loud — narrate your approach as if explaining to an interviewer
- 🖋️ Always test your solution manually with edge cases before submitting
- 📊 Analyze **time and space complexity** for every solution you write
- 🔄 Revisit problems you got wrong — second attempts reveal real understanding
- 📅 **New:** Keep a "mistakes log" — one line per problem noting what you missed. Review it every Sunday.

PHASE 1

Foundations

Days 1–10

Days 1–3 Arrays & Sorting Basics

CONCEPTS

- Array creation, indexing, slicing, traversal
- In-place manipulation vs. creating new arrays
- Multi-dimensional arrays
- Sorting algorithms: Bubble Sort → Insertion Sort → Selection Sort
- Analyze best / average / worst case for each sort

PRACTICE QUESTIONS

Two Sum

Best Time to Buy & Sell Stock

Move Zeroes

Rotate Array

Missing Number

Insight: Before jumping to Two Sum's hash map trick, actually implement the $O(n^2)$ brute force first. Feeling the difference in your bones makes the optimization click permanently.

Days 4–5 Stacks & Queues

CONCEPTS

- Stack: LIFO — push, pop, peek
- Queue: FIFO — enqueue, dequeue
- Circular Queue — implement from scratch using arrays
- Monotonic Stack (intro)
- Implement both using arrays and linked lists

PRACTICE QUESTIONS

[Valid Parentheses](#)[Min Stack](#)[Queue using Stacks](#)[Next Greater Element](#)

Days 6–10 Core Patterns — Arrays & Strings

PATTERNS TO MASTER

Pattern	When to Use
Two Pointer	Sorted arrays, pair-finding, palindromes
Sliding Window	Subarray/substring with a constraint
Subset / Combination	Generate all subsets or combinations
Merge Intervals	Overlapping ranges, scheduling
String Manipulation	Anagrams, reversal, character frequency

PRACTICE QUESTIONS

[Container With Most Water](#)[3Sum](#)[Longest Substring Without Repeating](#)[Subsets](#)[Merge Intervals](#)[Group Anagrams](#)[Longest Palindromic Substring](#)

Insight: These five patterns cover roughly 40% of all Easy/Medium interview questions on LeetCode. Master them here and Phase 2–5 will feel like variations on a theme rather than new material.

PHASE 2 Core Data Structures

Days 11–20

Days 11–13 Recursion & Backtracking

CONCEPTS

- Base case and recursive case — visualize the call stack
- Memoization as a bridge to DP
- Backtracking: explore → choose → unchoose
- Recognize exponential blowup → prune early

PRACTICE QUESTIONS

[Fibonacci \(iter + recursive\)](#)[Permutations](#)[Subsets II](#)[Letter Combinations](#)[Word Search](#)[N-Queens](#)

Insight: Draw the recursion tree on paper for at least 5 problems before trusting yourself to trace it mentally. Backtracking bugs are almost always a misunderstanding of when state gets "unchosen."

Days 14–16 Advanced Sorting & Binary Search

SORTING ALGORITHMS

- Merge Sort — divide & conquer, stable, $O(n \log n)$
- Quick Sort — partition, pivot selection, average $O(n \log n)$
- Count Sort — integer keys, $O(n + k)$
- Radix Sort — digit-by-digit, large integers

BINARY SEARCH PATTERNS

- Classic binary search on sorted arrays
- Search on answer space (not just arrays)
- First/last position of element
- Search in rotated sorted array
- Binary search on floating point answers

PRACTICE QUESTIONS

[Binary Search](#)[Search Rotated Array](#)[First & Last Position](#)[Koko Eating Bananas](#)[Find Peak Element](#)[Median of Two Sorted Arrays](#)

Insight: "Binary search on the answer" is one of the highest-leverage patterns for Medium/Hard problems. If a problem says "minimize the maximum" or "find the smallest X such that condition holds," suspect binary search on answer space immediately.

Days 17–20 Linked Lists

CONCEPTS

- Singly Linked List — traversal, insertion, deletion
- Doubly Linked List — bi-directional pointers, LRU Cache
- Circular Linked List — detect and handle
- Floyd's Cycle Detection (slow/fast pointer)
- Dummy node technique

PRACTICE QUESTIONS

[Reverse Linked List](#)[Merge Two Sorted Lists](#)[Merge K Sorted Lists](#)[Remove Nth From End](#)[Linked List Cycle](#)[Reorder List](#)[LRU Cache](#)

Days 21–25 Trees & Binary Search Trees

TREE TRAVERSALS

- In-order ($L \rightarrow \text{Root} \rightarrow R$) — gives sorted output for BST
- Pre-order ($\text{Root} \rightarrow L \rightarrow R$) — used for copying trees
- Post-order ($L \rightarrow R \rightarrow \text{Root}$) — used for deletion
- BFS / Level-order — using a queue
- DFS — using recursion or explicit stack

BST CONCEPTS

- Insert, search, delete, validate BST
- Balanced BSTs — know AVL concept, not full implementation

PRACTICE QUESTIONS

Max Depth

Diameter of Binary Tree

Invert Binary Tree

Symmetric Tree

Path Sum

Lowest Common Ancestor

Validate BST

Level Order Traversal

Serialize & Deserialize

Insight: Almost every tree problem reduces to "what do I compute at each node from my children's answers?" Practice stating that one sentence out loud before coding — it forces you to define the recursive return value clearly.

Days 26–28 Heaps & Priority Queues

CONCEPTS

- Min-Heap and Max-Heap — structure and properties
- Insert $O(\log n)$, extract $O(\log n)$, peek $O(1)$
- Heapify — build heap from array in $O(n)$
- Heap Sort — in-place, $O(n \log n)$
- Two-heap technique for median problems

PATTERN: TOP K ELEMENTS

K Closest Points to Origin

Top K Frequent Elements

Kth Largest Element

Find Median from Data Stream

Task Scheduler

Reorganize String

Days 29–30 Hashing & Advanced Structures

CONCEPTS

- Hash Maps and Hash Sets — collision handling internals
- When to use HashMap vs. array vs. set
- Prefix Sum — for range queries in $O(1)$
- Monotonic Stack/Queue — advanced applications

PRACTICE QUESTIONS

Group Anagrams

Top K Frequent Words

Subarray Sum Equals K

Sliding Window Maximum

Longest Consecutive Sequence

Insight: Prefix sums + hash maps together (like in Subarray Sum Equals K) are one of the most under-practiced combos. Whenever you see "count subarrays with sum/property X," check if a running prefix stored in a hash map solves it in $O(n)$.

Days 31–33 Graphs

REPRESENTATIONS

- Adjacency Matrix — $O(V^2)$ space, fast edge lookup
- Adjacency List — $O(V+E)$ space, preferred for sparse graphs
- Edge list

ALGORITHMS

- BFS — shortest path (unweighted), level-by-level
- DFS — cycle detection, connectivity, path finding
- Union-Find (DSU) — dynamic connectivity
- Dijkstra's — shortest path in weighted graph
- Bellman-Ford — handles negative weights
- Prim's / Kruskal's — Minimum Spanning Tree

PATTERN: TOPOLOGICAL SORT

- Only works on Directed Acyclic Graphs (DAGs)
- Kahn's Algorithm (BFS) or DFS-based

PRACTICE QUESTIONS

[Number of Islands](#)[Clone Graph](#)[Course Schedule I & II](#)[Pacific Atlantic Water Flow](#)[Word Ladder](#)[Network Delay Time](#)[Redundant Connection](#)

Insight: Graphs are where most candidates lose points not on algorithm knowledge but on setup speed. Practice writing an adjacency list from an edge list in under 60 seconds — it's a mechanical step that shouldn't eat interview time.

Days 34–35 Trie & Segment Trees

TRIE (PREFIX TREE)

- Insert, Search, StartsWith — implement TrieNode from scratch
- Used for: autocomplete, spell-check, prefix matching

SEGMENT TREE (CONCEPTUAL)

- Range queries (sum, min, max) in $O(\log n)$
- Know when to apply — full implementation is a bonus

PRACTICE QUESTIONS

[Implement Trie](#)[Word Search II](#)[Design Add & Search Words](#)[Replace Words](#)

Days 36–38 DP Foundations & Framework

6-STEP DP FRAMEWORK

- Identify if it's a DP problem (optimal substructure + overlapping subproblems)
- Define the state: what does $dp[i]$ or $dp[i][j]$ represent?
- Write the recurrence relation
- Identify base cases
- Decide: Top-Down (Memoization) or Bottom-Up (Tabulation)
- Optimize space if possible (rolling array, 1D reduction)

CORE DP PATTERNS

Pattern	Classic Problems
1D DP	Climbing Stairs, House Robber, Decode Ways
2D Grid DP	Unique Paths, Min Path Sum
Knapsack 0/1	Subset Sum, Partition Equal Subset
Unbounded Knapsack	Coin Change, Rod Cutting
LCS / Edit Distance	Longest Common Subsequence, Edit Distance
LIS	Longest Increasing Subsequence
Palindrome DP	Longest Palindromic Subsequence
String DP	Word Break, Interleaving Strings
Interval DP	Burst Balloons, Matrix Chain
State Machine DP	Buy/Sell with Cooldown

Insight: Every DP problem is secretly a recursive brute force with a memory. If you're stuck, write the brute-force recursion first (even if exponential), then add a memo dictionary keyed on the state. The "framework" only clicks after you've done this conversion by hand 8-10 times.

Days 39–42 DP Deep Dive Practice

DAILY METHOD (2–3 PROBLEMS/DAY FROM EACH PATTERN)

- Explain approach in words before coding
- Draw the DP table/tree for small examples
- Code it without looking at solution
- Review optimal solution after your attempt
- Re-implement from memory the next day

SCHEDULE

- Day 43: 2 × 45-min sessions (1 Easy + 1 Medium each)
- Day 44: 2 × 45-min sessions (2 Mediums each)
- Day 45: Full mock — 1 Easy + 1 Medium + 1 Hard, full narration

CHECKLIST FOR EVERY PROBLEM

- ☐ Clarify the problem (ask edge case questions)
- ☐ State your approach before coding
- ☐ Write clean code with meaningful variable names
- ☐ Trace through with an example
- ☐ Analyze time and space complexity
- ☐ Identify edge cases: empty input, single element, negatives
- ☐ Suggest follow-up optimizations

Insight: Record yourself (audio is enough) during at least one mock session. Most people underestimate how much they say "um" or go silent while thinking — a real interviewer reads silence as being stuck, not as thinking.

REFERENCE

Patterns Quick Reference

Pattern	Days	Signal to Use
Two Pointer	6–10	Sorted array, pair/triplet finding
Sliding Window	6–10	Contiguous subarray with constraint
Subset / Combination	6–13	Generate all possibilities
Merge Intervals	6–10	Overlapping ranges
Backtracking	11–13	Decision tree, undo choices
Modified Binary Search	14–16	Sorted or rotated array
Fast & Slow Pointer	17–20	Cycle detection, midpoint
Tree BFS	21–25	Level-by-level processing
Tree DFS	21–25	Path problems, subtree properties
Top K Elements	26–28	K largest / smallest / frequent
Topological Sort	31–33	Ordering with dependencies
Union-Find	31–33	Dynamic connectivity
Memoization / DP	36–45	Overlapping subproblems

Insight: Print this table and tape it above your desk. When you get stuck on a new problem during practice, scan this table first — before searching for a solution online. Pattern recall under pressure is the actual interview skill being tested.

Week	Focus	Goal
Week 1 (Days 1–7)	Arrays, Stack, Queue, Basic Patterns	15+ problems
Week 2 (Days 8–14)	Recursion, Sorting, Binary Search	15+ problems
Week 3 (Days 15–21)	Linked Lists, Trees intro	15+ problems
Week 4 (Days 22–28)	BST, Heaps, Hashing	15+ problems
Week 5 (Days 29–35)	Graphs, Trie, Advanced patterns	15+ problems
Week 6 (Days 36–42)	Dynamic Programming deep dive	20+ problems
Days 43–45	Mock interviews, review weak areas	Full simulations



BOOK

Cracking the Coding Interview

Gayle Laakmann McDowell — interview patterns & problems



BOOK

Introduction to Algorithms (CLRS)

Cormen et al. — deep algorithm theory



VIDEO

CS50 — Harvard

Foundations of computer science



VIDEO

A2Z DSA Playlist — Striver

Structured DSA from basics to advanced



VIDEO

Blind 75 — NeetCode

Top interview patterns explained



VIDEO

Abdul Bari — Algorithms

Deep algorithm theory with visualizations



PRACTICE

LeetCode

Primary practice platform — filter by tag/pattern



PRACTICE

NeetCode 150

Superset of Blind 75 — curated list with solutions

Remember: The goal isn't to memorize solutions. It's to build pattern recognition so you can construct solutions under pressure. Struggle productively — the discomfort of not knowing *is* the learning.



Thank You for Visiting!

I hope this roadmap gives you the structure and confidence to crack your next coding interview. Consistency beats intensity — show up daily, and the results will follow.

🌟 Do give a star & share with a friend who's prepping too!

Got feedback, found a bug in the roadmap, or want to share your progress? I'd genuinely love to hear it.

See you back here soon — happy coding! 🚀

Curated & compiled by **Vaibhav Singh (AIML)**